Criiq — Technical Documentation

Data Collection & Model Training: Architecture, Decisions, and Rationale

Sushma Shrestha

13 July 2026

Contents

Introduction and Project Overview **3**
- What Criiq is . . . 3
- Project timeline . . . 3
- Guiding design principles . . . 3
- Repository layout . . . 3

System Architecture **5**
- The two-module contract . . . 5
- End-to-end data flow . . . 5

Part I — The Data-Collection Module **6**
- Purpose and scope . . . 6
- Data sources and their roles . . . 6
  - Cricsheet (authoritative spine) . . . 6
  - Cricsheet people register . . . 6
  - ESPNcricinfo (best-effort metadata) . . . 6
  - Wikidata and Wikipedia (bowling styles, roles, squads) . . . 6
  - ICC rankings . . . 7
  - RSS feeds (retirement monitoring) . . . 7
  - What is deliberately NOT collected . . . 7
- Why SQLite (and not something else) . . . 7
  - The reasons for SQLite . . . 7
  - The alternatives and why they were not chosen . . . 7
- Database schema . . . 8
  - Core / raw tables . . . 8
  - Registry / status / metadata tables . . . 8
  - Derived analytics tables . . . 9
  - Outside SQLite (Parquet) . . . 9
- The pipeline: 12 stages, in order . . . 9
  - Conditional downloads . . . 10
  - Polite HTTP: rate limiting and retries . . . 10
- Ingestion, validation, and cleaning . . . 11
- Identity resolution — the hard problem . . . 11
  - The gender migration — a real upstream data quirk . . . 11
- Derived analytics layer . . . 13
  - Bowling containment stats . . . 13
  - The per-opponent matchup breakdown (`vs-team`) . . . 14
- The ML feature layer . . . 14
- Win probability and WPA . . . 14

The empirical win-probability model (`winprob.py`) . . . 14
Win-Probability-Added (`wpa.py`) . . . 14
The Python CLI — why it was built . . . 15
Testing (data-collection) . . . 15
The EDA notebooks . . . 15

Part II — The Model-Training Module **16**
Purpose and architecture . . . 16
Scaffolding decisions . . . 16
Feature engineering . . . 16
The feature schema . . . 16
How the features are computed . . . 17
Bayesian-style shrinkage . . . 17
How leakage is prevented (exactly) . . . 17
The style-matchup signal . . . 18
The target: why WPA and not contribution share . . . 18
Models: what is trained, what was tried, what won . . . 18
The two production models . . . 18
How training actually works — no epochs, no batches . . . 19
Fine-tuning attempt: adding an age feature, and why it wasn't kept . . . 20
Models tried — the full comparison . . . 20
Why the current models were selected . . . 21
Why the comparison is called "robust" . . . 21
Experiments tried and rejected (with reasons) . . . 22
Evaluation: metrics and validation strategy . . . 22
Metrics, defined . . . 22
Validation strategy: temporal holdout, not cross-validation . . . 23
The recommendation engine . . . 23
The composite score . . . 23
Team-need synergy . . . 24
Explanations . . . 24
XI selection as constrained optimization . . . 25
Performance note . . . 25
Testing (model-training) . . . 25

Strengths and Weaknesses — an Honest Assessment **26**
What the project does well . . . 26
What is weak or unfinished . . . 26
Resolved since the above was first written (kept for the record) . . . 27

What Can Be Done to Make It Better **28**

Appendix A — Dependencies and Why Each Exists **29**
data-collection . . . 29
model-training . . . 29

Appendix B — CLI Quick Reference **29**
data-collection (`python run.py <cmd>`) . . . 29
model-training (`python -m criiq_models.cli <cmd>`) . . . 30

Appendix C — Glossary **30**

Introduction and Project Overview

What Criiq is

Criiq is a cricket analytics and **player-selection recommendation system**. Given a fixture context — *"Team A is playing Team B (optionally at venue V)"* — it scores every eligible player in Team A's pool and proposes a balanced playing XI, with plain-language reasoning attached to every pick. The scope is **T20 Internationals, both genders** (`format`/`gender` are settings, never hardcoded): women's T20I was built first and remains the default when `--gender` is omitted, but men's T20I is now an equally trained, equally verified second scope — its own models, its own registry-status verification across nine national teams, its own end-to-end CLI checks — not just "ingested but unmodelled."

The system is split into two cooperating halves, developed in sequence:

1. `data-collection` — turns public cricket data (primarily Cricsheet ball-by-ball data) into a single, self-describing SQLite database (`criiq.db`) plus a handful of Parquet feature files. It handles ingestion, validation, identity resolution, enrichment, derived analytics, and ML feature preparation.
2. `model-training` (`criiq_models`) — a strict **read-only consumer** of that database. It engineers leakage-safe features, trains two Win-Probability-Added (WPA) regression models, evaluates them honestly against baselines, and wraps everything into a transparent recommendation engine with constrained XI selection.

A third component (`webapp`) exists in the repository but is outside the scope of this document.

Project timeline

The project began on **19 June 2026**. The data-collection module was built first (late June: Cricsheet ingestion, identity resolution, derived tables, registry, rankings, win probability, WPA, role finalization), followed by the model-training module (early–mid July: scaffolding, feature engineering, recommendation engine, evaluation, and the robust model comparison). Development proceeded in staged, well-scoped commits, each of which is reflected in a section of this document.

Guiding design principles

Several principles recur throughout both modules and explain most of the design decisions documented below:

- **One authoritative spine, best-effort enrichment.** Cricsheet ball-by-ball data is the source of truth for *events* (balls, lineups, results). ESPNcricinfo, Wikidata, and Wikipedia enrich *metadata* (date of birth, role, bowling style, retirement). The entire pipeline runs to completion on Cricsheet alone; every enrichment source can fail without breaking a build.
- **Deterministic joins over fuzzy matching.** Player identity is resolved through Cricsheet's people register, which carries the ESPNcricinfo ID (`key_cricinfo`). External data is merged by ID, never by guessing at names.
- **Nothing is fabricated.** Metrics with no free and legal data source (ball speed, shot-control percentages, wagon wheels, pitch maps) are deliberately excluded rather than approximated or faked.
- **Idempotent and incremental.** Every pipeline stage is safe to re-run; only new work is performed on each run.
- **Transparency over black boxes.** The recommendation score is a documented sum of parts, each traceable to raw numbers, so every recommendation can be explained in plain language.
- **Honest evaluation.** Models are always compared against a mean-predictor baseline, splits are temporal (never random-by-row), and weak results are reported as weak rather than dressed up.

Repository layout

```
criiq/
  data-collection/          # ingestion + feature engineering half
```

```
    criiq_data/           # 33 Python modules (pipeline, CLI, analytics)
    data/                 # criiq.db, raw ZIPs, Parquet features (gitignored)
    tests/                # ~45 offline unit tests
    criiq_overview.ipynb  # source-by-source data tour (50 cells)
    eda_heatmaps.ipynb    # statistical EDA companion (20 cells)
    run.py                # CLI entry shim
    README.md             # 430-line design document
model-training/           # AI / recommendation half
    criiq_models/         # features/, recommend/, selection/, evaluate/
    artifacts/            # trained model .joblib files + metadata
    tests/                # 10-file pytest suite on an in-memory DB
    model_comparison.ipynb        # original single-split comparison
    model_comparison_robust.ipynb # 3-fractions x 5-seeds comparison
    README.md             # engineering log with all decisions
webapp/                   # (out of scope here)
```

System Architecture

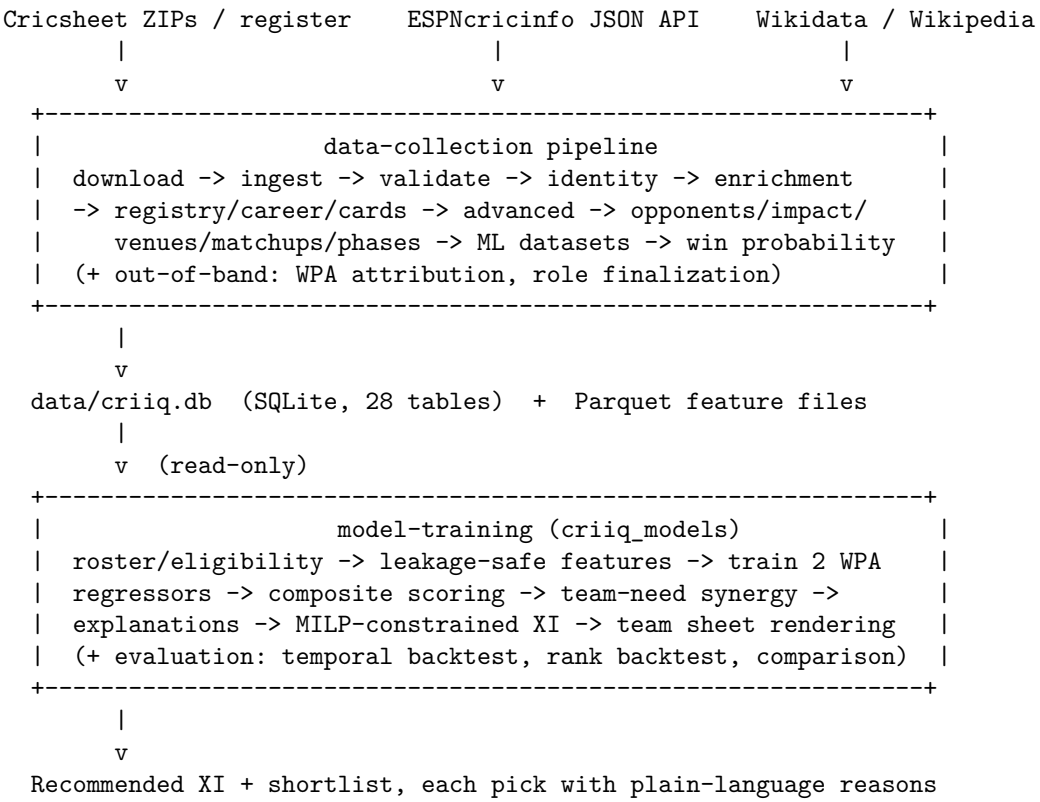
The two-module contract

The single most important architectural boundary in Criiq is the contract between the two modules:

- `data-collection` **owns and writes** `data-collection/data/criiq.db` (SQLite) and the Parquet feature files under `data-collection/data/features/`.
- `model-training` opens that database **read-only** (via a SQLite URI with `mode=ro`), never writes to it, and never imports `data-collection`'s Python code. It owns only its own derived artifacts (trained models and metadata JSON) under `model-training/artifacts/`.

This gives a clean, testable interface: the entire hand-off between the halves is *one file*. Either side can be rebuilt, tested, or replaced independently, and the model-training test suite runs against a tiny synthetic in-memory database that mimics only the columns it actually reads.

End-to-end data flow



Part I — The Data-Collection Module

Purpose and scope

`data-collection` converts public cricket data into a database that the rest of the system can trust. Its deliverable is `data/criiq.db`. As measured on the live database at the time of writing:

Quantity	Value
Database size	408 MB
Tables	28
Deliveries (balls)	1,226,642
Matches	5,428 (2,035 female T20I + 3,393 male T20I)
Players	7,259
Date range	2005-02-17 to 2026-06-15

Both genders are ingested, and both now have their own trained recommendation models — women's T20I remains the default scope, but men's T20I is fully modelled and verified as a second, equally-supported one (see "Extending to men's T20I" below).

Data sources and their roles

Cricsheet (authoritative spine)

- **What:** ball-by-ball CSV archives of T20 internationals (`t20s_female_csv2.zip`, `t20s_male_csv2.zip`) and the people register (`people.csv`).
- **Why:** Cricsheet is free, legal, complete at the ball level, and machine-readable. Every event-level fact in Criiq — every delivery, lineup, result — comes from here.
- **Why it is the authority on events:** it is the only source in the stack with per-ball granularity, and its per-match `registry` section provides the identifier join that makes deterministic identity resolution possible.

Cricsheet people register

Carries, for each player identifier, the canonical name, unique name, and crucially **`key_cricinfo`** — the ESPNcricinfo ID. This single column is what allows all external enrichment to be joined *by ID rather than by name*, eliminating almost all fuzzy matching. 100% of live players resolve with a `key_cricinfo`.

ESPNcricinfo (best-effort metadata)

ESPNcricinfo web pages actively block scraping (verified HTTP 403), so the pipeline uses only the ESPN **core JSON API** (`core.espnuk.org/v2/sports/cricket/athletes/{id}`), keyed by `key_cricinfo`. It fills date of birth, nationality, role, and batting hand where available. Responses are cached to Parquet so re-runs don't re-fetch. Enrichment is opt-in (`--enrich`) and the pipeline is fully functional without it.

Wikidata and Wikipedia (bowling styles, roles, squads)

Bowling style (pace vs spin, and finer buckets) is essential for the style-matchup signal in model-training but is absent from ball-by-ball data. It is seeded from Wikidata via SPARQL and enriched from Wikipedia infoboxes (parsed from wikitext). A manual CSV workflow (scaffold, fill by hand, load) covers what automation misses. This enrichment raised bowling-style coverage of real female T20 bowlers from 127 players (7.3%) to 568 (32.7%) — directly enabling the batter-versus-bowling-style features downstream. Wikipedia squad pages also supply the wicket-keeper flag, which does not exist in ball-by-ball data at all (Cricsheet's export omits the fielder of a dismissal).

ICC rankings

The ICC publishes only the **top 10** per discipline publicly; the full list sits behind a credentialed widget. The importer supports the public endpoint (gated on a `CRIIQ_ICC_RANKINGS_CLIENT_ID` environment variable — it explicitly refuses to guess or bypass credentials) and a pluggable CSV workflow. Rankings feed the registry's activity classification and a modest "standing" component in the recommendation score.

RSS feeds (retirement monitoring)

ICC and ESPNcricinfo RSS feeds are scanned for retirement keywords. Matches create **pending alerts** that a human confirms or dismisses via the CLI — retirement status is never changed automatically from a headline, because RSS text is far too noisy to act on unreviewed.

What is deliberately NOT collected

Ball speed, shot-control percentage, wagon wheels / preferred shots, pitch maps, line-and-length, and foot-movement data have **no free and legal source**. The project's explicit position is that these are excluded rather than faked or approximated. This boundary is documented in the README and the overview notebook so that no downstream consumer mistakes absence for oversight.

Why SQLite (and not something else)

The choice of SQLite is one of the most consequential decisions in the project, so its rationale deserves a full treatment.

The reasons for SQLite

1. **The database *is* the deliverable.** The entire contract between the two modules is one file. A single self-contained `criiq.db` can be copied, backed up, versioned, diffed against a backup (as was actually done for the gender migration), and handed to the model-training module or a grader with zero installation steps.
2. **Zero operational overhead.** No server process, no accounts, no ports, no configuration. The pipeline is "orchestration-agnostic": any scheduler — cron, Task Scheduler, CI — can just run `refresh` and point at the file.
3. **It is in the Python standard library.** `sqlite3` ships with Python; the ingestion half needs no database driver dependency at all.
4. **The scale fits comfortably.** 408 MB, 1.23 million delivery rows, 28 tables. SQLite handles this size trivially; indexes on the hot paths (match/striker/bowler lookups) keep derived-table builds fast.
5. **WAL mode gives concurrent reads during writes.** The database is opened in Write-Ahead Logging mode, so notebooks and the model-training CLI can read while a refresh is writing.
6. **Read-only enforcement is trivial.** Model-training opens the file via a `file:...?mode=ro` URI — the *engine itself* guarantees the consumer cannot corrupt the producer's data. That guarantee is much harder to arrange casually with a server database.
7. **Real relational features where they matter.** Upserts (`ON CONFLICT DO UPDATE`), unique constraints (which are load-bearing here — see the gender migration), foreign keys, and indexes — none of which flat CSV files provide.

The alternatives and why they were not chosen

- **PostgreSQL / MySQL:** a server database would add installation, credentials, networking, and hosting concerns to an academic project with exactly one writer and a handful of readers. Nothing in the workload (single-writer batch pipeline, read-heavy consumers) needs a server's concurrency model. The costs are real; the benefits would be unused.
- **CSV files only:** no joins, no indexes, no constraints, no upserts, no transactional integrity. The identity-resolution and derived-table layers depend on all of these. CSVs do appear in the project, but

only as *human-editable exchange formats* (manual styles, overrides, rankings scaffolds), which is the role they are good at.

- **Parquet only:** columnar files are excellent for large analytical scans but poor as a transactional store of record — no upserts, no unique constraints, no incremental point updates. The project uses a **hybrid**: SQLite as the store of record and Parquet precisely where columnar wins — the per-ball ML datasets (`ball_state_T20.parquet`, `win_prob_T20.parquet`) and fitted win-probability grids, which are large, write-once, and read as whole columns.
- **DuckDB:** a strong analytical engine, but the pipeline's needs are transactional (incremental upserts, constraints, migrations) more than analytical throughput, and SQLite's ubiquity (stdlib support, tooling, DB Browser, etc.) made it the lower-risk choice. Where analytical performance matters, Parquet + pandas already covers it.

One related stack decision recorded in the code: **Pandera** (a dataframe-validation library) was tried and intentionally removed in favour of lightweight hand-rolled pandas validation, to keep the stack reliable on Python 3.13.

Database schema

The schema (defined in `db.py` as a single `SCHEMA` script of `CREATE TABLE IF NOT EXISTS` statements) groups into four layers.

Core / raw tables

Table	Purpose
`meta`	Pipeline bookmarks: download fingerprints, `last_successful_refresh`, per-stage build timestamps.
`matches`	One row per match: format, gender, teams, toss, winner, margin, venue, player of match.
`deliveries`	One row per ball (1.23 M rows): innings, over, ball, batter, bowler, runs, extras, wicket info.
`match_players`	XI membership — which players appeared in which match for which team.
`identifier_map`	Player display-name to Cricsheet identifier pairs, parsed verbatim from each match file.
`people`	The Cricsheet register: identifier, names, `key_cricinfo`.
`players`	The resolved player entity: stable `player_id` (UUID5), gender, canonical name, `key_cricinfo`, role, resolution method. Constraint: `UNIQUE(cricsheet_identifier, gender)`.
`venues`	Raw venue string to canonical venue mapping.

Registry / status / metadata tables

Table	Purpose
`player_registry`	The eligibility engine's output: retirement/activity status, last appearance, age, career phase, recall flag, selection rate, ICC rank.
`icc_rankings`	Imported ICC top-10 rankings per discipline.
`player_styles`	Bowling style, batting hand, playing role (from Wikidata/Wikipedia/manual CSV).

Table	Purpose
status_overrides	Manual, human-confirmed status corrections.
retirement_alerts	Pending/confirmed/dismissed RSS retirement alerts.
discrepancies	Scaffolded Cricsheet-vs-Cricinfo mismatch log (currently unused).

Derived analytics tables

Table	Purpose
player_career	Career batting and bowling aggregates per player per format.
player_vs_opponent	Full record against each opponent (matches, win %, batting, bowling, player-of-match).
player_impact	Contribution shares, performance in wins vs losses, pressure splits, and a transparent weighted importance_score with JSON components.
player_phase	Powerplay / middle / death splits for batting and bowling, with intra-team rank and percentile.
player_by_venue	Per-ground batting and bowling records.
dismissal_matchup	Batter-versus-bowler head-to-head samples (balls, runs, dismissals) — 103,080 rows live.
player_advanced	JSON profile: phase splits, control/aggression proxies, acceleration, archetype (anchor/power hitter/finisher/balanced), vs-pace/spin and vs-handedness splits, plain-language strengths.
player_cards	A nested dashboard blob per player (identity, career, form index, impact, status, ranking, recent H2H).
player_role_final	Refined primary role, keeper flag with confidence and source, median batting position.
player_match_impact	Per-match batting and bowling contribution, including dot-ball %, boundary %, and bowling strike rate.
player_match_wpa	Per-match batting and bowling Win-Probability-Added.
player_phase_wpa	Career per-phase WPA rates.
match_features	Leakage-safe pre-match features and result label for match-level prediction.

Outside SQLite (Parquet)

- ball_state_T20.parquet — per-ball game state plus win label; player-agnostic training data for the win-probability model.
- win_prob_T20.parquet — every ball scored with its win probability.
- data/models/winprob_T20_*.parquet + meta JSON — the fitted empirical win-probability grids.

These are columnar, write-once, whole-column-read artifacts — exactly the access pattern Parquet is designed for, which is why they are not tables.

The pipeline: 12 stages, in order

The orchestrator (refresh.py) runs the following stages, each idempotent and individually re-runnable via the CLI:

#	Stage	What it does	Source
1	register	Download the Cricsheet people register; upsert `people`.	cricsheet.org
2	download	Conditionally download the T20I ball-by-ball ZIPs.	cricsheet.org
3	ingest	Parse each match's info + deliveries CSVs; validate; write raw tables.	local ZIPs
4	identity	Resolve players to stable IDs; normalise venues.	DB only
5	cricinfo enrichment	(Opt-in) fill DOB/nationality/role from ESPN JSON API.	core.espnuk.org
6	styles	Merge manual styles CSV; optionally seed from Wikidata/Wikipedia.	Wikidata/Wikipedia
7	RSS scan	Scan retirement feeds into pending alerts.	ICC/ESPN RSS
8	registry + career + cards	Rankings, overrides, activity registry, career stats, player cards.	DB / ICC
9	advanced profiles	Build archetypes and advanced JSON profiles.	DB
10	per-opponent, impact, venue, matchups, phases	All remaining derived analytics tables.	DB
11	ML datasets	`match_features`, ball-state Parquet, `player_match_impact`.	DB
12	win probability	Fit the empirical WP model; score every ball.	Parquet

Two heavyweight stages are deliberately **out-of-band** (run as standalone CLI commands, not inside `refresh`): **WPA attribution** (`wpa`) and **role finalization** (`role-finalize`). This is a known operational caveat — a scheduler running only `refresh` will leave `player_match_wpa`, `player_phase_wpa`, and `player_role_final` stale (see Limitations).

Conditional downloads

Downloads are fingerprint-gated: if the ZIP already exists, an HTTP `HEAD` request compares `"{Last-Modified}|{Content-Length}"` against a stored fingerprint in `meta`. Unchanged files are skipped entirely. Changed files stream to a `.part` temp file in 64 KB chunks and are atomically renamed into place, so an interrupted download can never corrupt a good archive.

Polite HTTP: rate limiting and retries

All network access goes through one shared `requests.Session` (module `http.py`) with:

- A project-identifying User-Agent (`criiq-data-collector/0.1 (academic project; ...)`).

- **Per-host politeness delays** — a monotonic-clock map guarantees a minimum gap between requests to the same host: 1.0 s for Cricsheet, 2.0 s for ESPN, 1.5 s for Wikidata SPARQL, 1.0 s for the Wikipedia API.
- **Retries with exponential backoff** — up to 3 attempts, backing off 2 s then 4 s, on HTTP 429/500/502/503/504 and connection errors; 120 s timeout.

This is needed because the pipeline touches other people's free infrastructure; being a good citizen (and not getting IP-banned mid-semester) is both an ethical and a practical requirement.

Ingestion, validation, and cleaning

Parsing (`ingest.py`). Each match archive contains an `_info.csv` (metadata as `info,key,value...` rows — hand-parsed with a real CSV reader because venue names contain commas) and a deliveries CSV. Dates are normalised to ISO; win margins are derived ("N runs" / "N wickets" / method). The ball identifier is split into over and ball-in-over; run totals are computed; NaNs are converted to None.

Scope filtering. Only `T20`/`IT20` match types with `team_type = "international"` are ingested — franchise and domestic games are excluded by design so that "current team" derivation and eligibility logic operate on national sides only.

Validation (`validate.py`). Every deliveries frame must contain the required columns; numeric fields are coerced (rows with non-numeric values are rejected); rows with null batter/bowler/team fields are rejected. Rejected rows are not silently dropped — they are written with a `_reject_reason` to a timestamped **quarantine CSV** (`data/quarantine/`), so data-quality problems are visible and auditable.

Deduplication and incrementality. Matches already present are skipped (`_existing_match_ids`); raw inserts use `INSERT OR IGNORE`; every derived table uses upserts. Re-running any stage is safe.

Identity resolution — the hard problem

Attributing 1.2 million balls to the right human beings is the module's core correctness problem. The solution has several layers:

1. **Deterministic register join.** Each match file lists `(display name, cricsheet identifier)` pairs. Identifiers join to the `people` register, which supplies `key_cricinfo`. Every player gets a stable `player_id` = UUID5 of `"cricsheet:{identifier}"`, a resolution method (`register` / `register_no_cricinfo` / `cricsheet_only`), and a `needs_review` flag when no Cricinfo key exists.
2. **External merges strictly by ID.** Styles, roles, and enrichment join on `key_cricinfo`, never raw names — so a Wikipedia "Smriti Mandhana" squad entry can never mint a duplicate of Cricsheet's "S Mandhana".
3. **Fuzzy matching as a last resort only.** RapidFuzz `WRatio` with a threshold of 88 (90 for rankings/overrides) exists as a fallback, but the deterministic join makes it essentially unnecessary in practice.
4. **A purpose-built name heuristic where fuzzy fails.** Wikipedia lists "First Last" while Cricsheet uses "XY Surname"; generic fuzzy scoring puts such pairs *below* threshold (~85) even when they are the same person. `match_full_name_to_cricsheet` instead matches by surname + leading initial, scoped to a single team's roster, and reports ambiguity rather than guessing.
5. **Derived roles from behaviour.** From legal-ball counts: 60+ balls faced *and* 60+ bowled = all-rounder; 60+ bowled = bowler; any faced = batter — later refined by `role_finalize` (below).
6. **Venue normalisation.** Hardcoded aliases for known spelling variants (Brabourne, Wankhede, Chinnaswamy, PCA Mohali) plus a generic "take the part before the first comma" rule, slugged into a `venue_id`.

The gender migration — a real upstream data quirk

The single most instructive correctness episode in the project: **Cricsheet reuses 32 identifiers across two genuinely different real people of different genders** (e.g. "K Joshi" is both a Nepal women's

player and an unrelated men's player; "A Sharma" both a female India player and a male UAE player). This is an upstream quirk, not a matching bug — but once both genders were ingested, any table keyed on identifier alone would silently blend two people's statistics.

The fix was structural:

- The `players` table's constraint changed from `cricsheet_identifier UNIQUE` to **`UNIQUE(cricsheet_identifier, gender)`**. SQLite cannot alter a unique constraint in place, so the migration rebuilds the table (create new, copy, drop, rename, re-index) — the pre-migration database was backed up as `criiq.db.pre-gender-migration-backup`, which made the delta independently verifiable.
- The ~99.5% of unambiguous identifiers keep `gender = NULL` and their **exact original `player_id`** (zero churn for the common case). Each of the 32 colliding identifiers mints two stable IDs (UUID5 of `"cricsheet:{identifier}:{gender}"`), one per gender. Verified live: 7,259 players = 7,227 pre-migration + 32.
- **Every derived-table builder was reworked** to group by `(identifier, gender)` and resolve identity gender-safely — career, impact, phases, matchups, advanced, opponents, venues, registry, roles, WPA, and ML features. Roughly 2,842 orphaned blended rows were purged from derived tables when this rolled out.
- A later commit fixed a *second* instance of the same class of bug: the team-roster join in `role_finalize.team_candidates` was not gender-qualified, so a split identifier could match both people for one appearance.

~~**Known residual gap**~~ — **fixed.** A later, dedicated re-audit (prompted by a direct request to re-verify "men's and women's data are never mixed, for all country, and venue analysis is correct") found and fixed four more real bugs the original migration missed:

- **`player_card.py`** grouped by raw identifier alone (no gender in the key at all) — for all 32 colliding identifiers the female player's card was silently absent entirely, and the male card's `matches_played`/career/form/impact JSON was the straight sum of both real people's careers. Confirmed live: "SR Taylor"'s card showed `matches_played=103` (22 true male + 81 true female, blended) before the fix.
- **`h2h.py`'s `names_for_player`** resolved by raw display name via `identifier_map` (which has no gender column) with no gender filter, so any H2H/matchup lookup on one of the 32 names silently summed both real people's deliveries regardless of which player_id was queried. Confirmed live: real "A Sharma" (India women's vs. an unrelated UAE men's player) returned an identical blended 101 balls/108 runs for *either* player's id, instead of the true 27/24 (female) and 74/84 (male) splits.
- **The win-probability model itself (`winprob.py`)** was fit on both genders' balls blended together, with gender never used as a conditioning variable — the most foundational of the four, since `player_match_wpa`/`player_phase_wpa`/`player_domestic_form` (and therefore every WPA-based feature and training target model-training uses) all derive from it. Fixed by fitting/persisting a separate `EmpiricalWinProb` per `(format, gender)`. Real, now genuinely distinct calibration: female Brier 0.1557 vs. baseline 0.2447; male Brier 0.1727 vs. baseline 0.25. Both packages' models were retrained after this fix since the WPA labels themselves changed.
- **`styles.py`'s `bowler_buckets()`/`batter_hands()`** had the same gender-blind lookup pattern — dormant (zero of the 32 identifiers currently have both genders' style data populated at once) but fixed pre-emptively before ongoing Wikipedia enrichment could trigger it.

All four are fixed the same way every other builder already was: threading `gender` through the resolution key. `player_career`, `player_impact`, `player_phase`, `dismissal_matchup`, `player_advanced`, `player_by_venue`, `player_registry`, `player_match_wpa`/`player_phase_wpa` were all independently re-verified clean across all 32 colliding identifiers as part of the same audit — and, separately, zero `player_id` overlap was confirmed between men's and women's eligible pools across **all 89 countries** with both genders' T20I history in the live database.

A related, separately-confirmed bug from the same audit: **venue conflation.** `identity.py`'s `normalise_venues()` generic fallback (`raw.split(",")[0]`) had silently merged "County Ground" across 5 different real English grounds (54 matches) and "National Stadium" across New Zealand's

Hamilton ground and Pakistan's Karachi ground (29 matches). Fixed with explicit disambiguating aliases; the genuinely-unresolvable bare names now honestly surface their own "(ground unspecified)" bucket and a full "did you mean" list instead of guessing.

Derived analytics layer

All derived builders consume a shared per-(player, match) innings frame built once by `stats.py` (batting runs/balls, bowling balls/runs/wickets/dots/boundaries, out flag, team, opponent, result, date, gender, venue). Highlights:

- **`player_impact` and the importance score.** A transparent weighted score: contribution share (up to 35) + winning contribution (up to 25) + standout/player-of-match (up to 25) + a bowling bonus (up to 15, split **equally** between top-order wicket-taking rate and economical-spell rate — economy previously counted for nothing at all, a real imbalance fixed by calibrating an economy sub-bonus against the real per-innings economy distribution the same way every other threshold in this codebase is calibrated) + a batting-innings-worth-having bonus (up to 15), stored alongside a JSON breakdown of every component. This "score as documented sum of parts" pattern was later adopted wholesale by the recommendation engine.
- **`player_phase`.** Powerplay (overs 1–6) / middle (7–15) / death splits, with each player's *rank and percentile within her own team* per phase (minimum 30 balls) — the raw material for the model's phase-fit component and team-need synergy.
- **`player_advanced`.** A JSON profile with control/aggression proxies (boundary/dot/rotation percentages), six rate (sixes per 100 balls), acceleration (first 10 balls vs after), per-phase batting splits, an archetype label, and batter-vs-bowling-style splits (vs pace/spin, and finer buckets) — the latter only possible because of the Wikipedia style enrichment. The **archetype** (anchor / power hitter / finisher / balanced) is classified against *each gender's own* population percentiles, not a shared absolute bar — men's and women's T20 six-hitting and strike-rate distributions differ sharply (men's six-rate 90th percentile ~8.0 per 100 balls vs women's ~1.9), so an absolute threshold would misclassify wholesale. A power hitter is six-rate-driven (clears the rope well above the gender norm at a high strike rate); a finisher needs a real death-overs sample that is genuinely destructive (death strike rate *and* boundary% both above the gender's high bar); a minimum-sample gate (60 balls, 40 at the death for a finisher) blocks tiny-sample false positives (a real one: a bowler tagged "finisher" off 7 balls before the gate).
- **`player_registry`.** The eligibility engine. Classifies each player as ACTIVE / INCONSISTENT / INACTIVE / UNKNOWN / RETIRED_FORMAT / RETIRED_ALL / UNAVAILABLE from ICC rank presence, appearance recency (windows: 180 days recent, 540 active, 1095 inactive), and selection rate within the team's recent matches. Also computes career phase, age advisories (38/40), and a `recall_flag` for recently-returned players. Live status distribution: 2,312 INACTIVE, 1,998 ACTIVE, 1,601 INCONSISTENT, 1,307 UNKNOWN, 21 RETIRED_ALL, 14 UNAVAILABLE, 6 RETIRED_FORMAT — the RETIRED/UNAVAILABLE counts reflect a real, researched `status_overrides` sweep across nine national teams (India, Pakistan, England, Australia, Sri Lanka, Bangladesh, Nepal, Oman, USA), both genders, done specifically because pure appearance-recency can't infer an explicitly-announced retirement (e.g. Rohit Sharma and Virat Kohli both retired from T20Is on 2024-06-29, but recency alone left Rohit Sharma showing UNKNOWN — the exact kind of gap this override mechanism exists to close).
- **`role_finalize`.** Fixes a systematic bias in behaviour-derived roles: specialist bowlers accumulate tail-end batting balls and get mislabelled all-rounders. Median batting position (threshold: position 8) separates genuine all-rounders from tailenders. Wicket-keeper flags merge in from Wikipedia squad pages by `key_cricinfo`, with confidence and source recorded.

Bowling containment stats

Per-match bowling signal was originally far thinner than batting signal (a bowler's line in one match was little more than wickets and economy). Dot-ball %, boundary-conceded %, and bowling strike rate were extracted per match into `player_match_impact`. A subtle correctness detail: bowling strike rate for a wicketless spell is stored as `None`, not 0 — a zero would read as "instant wickets" when averaged; a null correctly reads as

"no signal".

The per-opponent matchup breakdown (`vs-team`)

The most recent data-collection work generalized batter-versus-bowler head-to-heads into per-opponent breakdowns:

- **Threshold redefinition.** `dismissal_matchup` originally kept only pairs with at least one dismissal — which made a bowler who safely *contained* a batter for 50 balls invisible. The filter is now **at least 6 balls faced** (about an over) regardless of dismissals, growing the table from ~54k to over 103k rows. Zero-dismissal rows are the point: they distinguish a "safe" bowler from a "bogey" bowler.
- `batter_vs_team_bowlers` — for a batter against a named opponent: every bowler faced 6+ balls with runs, balls, strike rate, dismissal rate; the **bogey bowler** is ranked by dismissal count first (3 dismissals in 80 balls outranks 1 in 6).
- `bowler_vs_team_batters` — the symmetric view for a bowler, with two deliberately different highlights: **best matchup** (batter dismissed most often) and **most expensive** (batter who has scored most). Verified example: Ellyse Perry vs India — best matchup Shafali Verma (4 dismissals), most expensive Jemimah Rodrigues (44 runs).

An all-rounder gets both sections; a specialist gets one; sides with no data emit an explicit "no recorded matchup data" note instead of empty tables.

The ML feature layer

Three artifacts prepared specifically for modelling:

1. `match_features` — one row per match with *pre-match-only* information: rolling-10-match form (win %, average score, average conceded), head-to-head win %, toss, batting-first — plus the result label. Leakage-safe by construction.
2. `ball_state_T20.parquet` — every ball reduced to player-agnostic game state (innings, score, wickets, balls remaining, runs required) plus the eventual match result: the training set for win probability.
3. `player_match_impact` — per-match per-player contribution measures, the base for both the WPA join and model-training's feature builders.

Win probability and WPA

The empirical win-probability model (`winprob.py`)

A deliberately simple, dependency-free model: historical win *frequencies* in binned game states (separate grids for setting vs chasing; bins of 6 balls, 5 runs / 10 runs, required-rate cap), with shrinkage toward a parent grid for sparse cells and a coarse-grid backoff. Design points:

- **Calibration is held out by match, never by ball** (a CRC32 hash of the match ID buckets whole games) — balls within a game are heavily correlated, so a ball-level split would flatter the model.
- Reported quality: **Brier score ~0.17** versus 0.25 for a coin-flip baseline; **log-loss ~0.51** versus 0.69.
- Known v1 limitations (documented in code): the curve is step-shaped due to coarse bins and is conditions-agnostic (no team-strength prior). The `fit`/`predict` surface was designed so an sklearn model can drop in behind it later.

The CLI can query it three ways: build it, replay a full match timeline ball by ball, or evaluate an arbitrary hand-written state (`--state "innings=2,balls_remaining=30,wickets=4,runs_required=40"`).

Win-Probability-Added (`wpa.py`)

WPA attribution replays every ball, computes the change in win probability it caused, and credits the **striker** with +delta and the **bowler** with −delta (the non-striker gets nothing — standard convention, and Cricsheet doesn't record which end a run-out involved). Per-ball credits aggregate to `player_match_wpa` (per player

per match — this becomes the model-training **label**) and `player_phase_wpa` (career per-phase rates — this powers team-need synergy).

WPA matters because it measures *whether a performance moved the team toward winning*, not just how big the numbers were. A rapid 30 in a tight chase can be worth more than a slow 60 in a dead game — contribution percentages cannot see that difference; WPA can.

The Python CLI — why it was built

The CLI (`criiq_data/cli.py`, exposed as `python run.py <command>` or `python -m criiq_data.cli <command>`) is the module's entire operational interface. It exists because the pipeline needed to be:

- **Reproducible** — one command (`refresh`) rebuilds everything deterministically; no notebook state, no manual steps to forget.
- **Granular** — every one of the 12 stages is also an individual command (`ingest`, `identity`, `career`, `matchups`, …), so a change to one builder doesn't force a full rebuild during development.
- **Schedulable** — a plain command with JSON output and an exit code is what cron / Task Scheduler / CI can run. This is the practical meaning of "orchestration-agnostic".
- **Human-in-the-loop where automation is unsafe** — retirement alerts are confirmed or dismissed by hand (`alerts --confirm N`); manual data enters through scaffold-fill-load CSV workflows (`styles --scaffold`, `rankings --scaffold`, `status-override --scaffold`, `role-finalize --load-external`), never by editing the database directly.
- **Inspectable without notebooks** — `status` (table counts, freshness, coverage), `card`, `profile`, `sheet` (a full selector one-pager), `h2h`, `vs-team`, and `winprob` make the database's contents interrogable from a terminal.

It is built on **argparse** (standard library — consistent with the module's minimal-dependency stance; no click/typer dependency for what subparsers already do). Each subcommand opens a connection, ensures the schema, does its work, and prints indented JSON. A five-line `run.py` shim keeps the invocation short.

Testing (data-collection)

About 45 offline unit tests (`tests/test_pipeline.py`) run against a synthetic in-memory/temporary database — no network, no real archive needed. Coverage includes: player-ID stability, the identity join and `needs_review`, derived roles, H2H math, registry status transitions and the recall-flag guard, win-probability learning/calibration/save-load round-trips, WPA striker-credit and bowler-debit, cards, per-opponent records, impact, venues, dismissal matchups including the vs-team safe-matchup behaviour, phase ranks, all three ML artifacts, rankings (CSV and mocked feed), overrides, style classifiers and Wikipedia infobox parsing, advanced profile splits, the surname-based external-role merge, validation rejection, and — critically — the **cross-gender collision split** regression test.

The EDA notebooks

- `criiq_overview.ipynb` (50 cells) — the guided tour: data sources, database shape, identity/matching quality, batting/bowling insights, registry and eligibility, the selector sheet, ground and bogey-bowler deep-dives, prediction datasets, and an explicit "what is NOT in the data and why it is not faked" section.
- `eda_heatmaps.ipynb` (20 cells) — the statistical companion. Its central discipline: **every analysis splits female vs male, never blended** (it rebuilds the player-gender lookup exactly the way the internal builders do). Sections: per-table data-completeness heatmaps by gender; an explanation of *why* the gaps exist (a null batting average means the player never batted — structural missingness from specialists, not dirty data, and imputing it would be wrong); correlation heatmaps across ~15 career/impact metrics per gender; top-15 batter x top-15 bowler dismissal heatmaps; phase strike-rate heatmaps; team-vs-team win-rate matrices; and distributions of strike rate and economy split by gender. Its closing caveat is worth repeating: these are descriptive statistics, and correlation is not predictive power — the honest predictive results live in the model-comparison notebooks.

Part II — The Model-Training Module

Purpose and architecture

`criiq_models` answers one question: *given a team, an opponent, and optionally a venue, who should play, and why?* Its pipeline:

```
criiq.db (read-only)
  -> roster.py            eligible player pool per team
  -> features/            leakage-safe training table (historical.py)
                          live per-player feature vector (current.py)
  -> recommend/train.py   fit + persist 2 WPA regressors
  -> recommend/score.py   composite 0-100 score from model + rule parts
  -> recommend/team_need  pool-level synergy bonus
  -> selection/optimize   MILP-constrained XI + batting-order presentation
  -> recommend/explain    every component -> coach-facing sentences
  -> recommend/render     text team sheet
  -> evaluate/            temporal backtest + rank backtest
```

Everything tunable lives in a frozen `Settings` dataclass (`config.py`), each value with a docstring justifying it — default scope (`format="T20"`, `gender="female"` — a default, not the only supported value: every CLI command and every function down to `RecommendationScorer` itself takes an explicit `gender` override, and men's T20I is trained and verified as an equally-supported second scope), the cold-start gate (10 career matches), the form window (15 matches, geometric decay 0.9), shrinkage prior strength (8), the all-rounder secondary-discipline discount (0.4), WPA-to-points scale (450), XI constraints, and the backtest fraction (0.2). Environment variables (`CRIIQ_DB`, `CRIIQ_MODEL_ARTIFACTS`) override paths.

Scaffolding decisions

- **Read-only DB access (`db.py`).** One `connect()` function opening the SQLite file with `mode=ro` and a Row factory. Every function takes a connection parameter — which is exactly how the test suite injects its in-memory fixture database.
- **CLI (`cli.py`).** Argparse again, eight commands: `features`, `train`, `backtest`, `rank-backtest`, `teams`, `recommend` (with `--team`, `--opponent`, `--venue`, `--explain`, `--pretty`), `explain` (single-player component breakdown, by ID or fuzzy name), and `analyze-xi` (scores and checks a coach's own hand-picked XI against the same constraints `recommend`'s XI is held to, plus a direct diff against the provably-optimal one). Every command takes `--gender` (default `female`) alongside `--format` — see `docs/cli-reference.md` for the full command-by-command reference. One pragmatic hack: on import it reconfigures stdout/stderr to UTF-8, because Windows consoles default to legacy code pages that mangle the explanation text.
- **Roster and eligibility (`roster.py`).** `player_registry.nationality` turned out to be entirely NULL for T20, so "which team does this player currently represent" is derived from **actual match appearances**: a player's current team is the team of her most recent appearance. Eligibility then hard-excludes RETIRED players and excludes INACTIVE ones unless recalled. The module maintains its own small gender-aware identifier map (mirroring, not importing, data-collection's resolver) to stay safe against the 32 collision identifiers.

Feature engineering

The feature schema

A single source of truth (`features/schema.py`) defines the exact columns shared by the training builder and the live builder, so the model's inputs always line up between train and serve. **22 numeric features + 1 categorical:**

Group	Features
Career (expanding, pre-match)	`career_matches`, `career_bat_contribution_pct`, `career_bowl_contribution_pct`, `career_bat_sr`, `career_bowl_economy`, `career_batting_wpa`, `career_bowling_wpa`, `career_bowl_wickets_per_match`
Recent form (15-match window, recency-decayed)	`form_bat_contribution_pct`, `form_bowl_contribution_pct`, `form_bat_sr`, `form_bowl_economy`, `form_batting_wpa`, `form_bowling_wpa`, `form_bowl_wickets_per_match`
Versus this opponent (expanding, shrunk)	`vs_opp_matches`, `vs_opp_bat_contribution_pct`, `vs_opp_bowl_contribution_pct`, `vs_opp_win_pct`
At this venue (expanding, shrunk)	`venue_matches`, `venue_bat_contribution_pct`, `venue_bowl_contribution_pct`
Categorical	`role_bucket` (batter / bowler / all-rounder / unknown)

Targets: **`batting_wpa`** and **`bowling_wpa`** — the player's per-match Win-Probability-Added from `player_match_wpa`.

How the features are computed

- **Career features** are expanding means over all *strictly prior* matches: sort by date, `shift(1)`, then `expanding().mean()`. `career_matches` is a count of prior matches.
- **Form features** apply the same shift, then a 15-match rolling window in which values are weighted by geometric recency decay (0.9 per step back), skipping missing values.
- **Opponent and venue features** are expanding means restricted to prior matches against that opponent / at that venue, then **shrunk toward the player's career baseline** (see below).
- **`career/form_bowl_wickets_per_match`** mask out matches where the player didn't bowl (`wickets.where(balls_bowle > 0)`) — the source table stores 0 (not NULL) wickets even for pure batters, and without the mask the feature dilutes toward zero. This one fix measurably improved bowling-WPA correlation (~0.09 to ~0.10).
- **`role_bucket`** prefers the refined `player_role_final.primary_role`, falls back to the registry role, and normalizes noise (a stray Wikidata "goalkeeper" collapses to "unknown"). Training and serving must, and do, resolve roles identically.

Bayesian-style shrinkage

Sparse records are the norm: a player has often faced an opponent 0–2 times. The shared helper `shrink(observed, n, prior, k=8)` computes `(n*observed + k*prior) / (n + k)` — with no data it returns the career prior; with lots of data it converges to the observed value. This matters twice over:

1. **Statistically** — a 1-match record against an opponent shouldn't swing a prediction.
2. **Distributionally** — a live query with no venue history would otherwise produce `venue_matches = 0` alongside a *missing* mean, a combination that **never occurs in training** (every training row's match had a real venue), which was confirmed to push predictions wildly out of distribution. Shrinkage makes zero-sample records degrade smoothly to "assume her career average".

How leakage is prevented (exactly)

Data leakage — letting information from a row's own match or future leak into its features — would make backtest numbers meaningless. Three mechanisms prevent it:

1. **Point-in-time reconstruction, not aggregate reuse.** The database's aggregate tables (`player_career`, `player_vs_opponent`, …) include *all* matches, including the one being predicted and every one after it. The training builder therefore never uses them: it **rebuilds every feature as-of each match** from per-match rows, with `shift(1)` applied before any expanding/rolling operation, so a row's own match can never enter its own features. Unit tests assert this literally (e.g., row N's career mean equals the mean of rows 1..N-1 only).
2. **Temporal splits, never row splits.** Evaluation cuts by **match date quantile** — train strictly before the cutoff, test at/after. Random row-level splits are explicitly rejected because rows from the same match are correlated (teammates share a result), which would leak outcome information across the split.
3. **A deliberate non-feature.** The batter-vs-bowler `dismissal_matchup` table is career-total with no per-match granularity, so it *cannot* be reconstructed as-of a past match. Rather than accept leakage, it is excluded from model features entirely and used only at live scoring time as a transparent rule-based component.

The live builder (`current.py`) mirrors the same feature shape but uses the player's *entire* history — correctly, since a live recommendation predicts a genuinely future match.

The style-matchup signal

A secondary signal capped at ±5 points, answering "how does this batter handle the kind of attack this opponent bowls?":

1. `opponent_bowling_mix` computes the opponent's current pace/spin share from `player_styles` (requires at least 2 bowlers with known styles, else returns nothing).
2. `style_matchup_edge` reads the batter's per-style dismissal record from her advanced profile, shrinks each style's dismissal rate with a strong prior (strength 48 — this data is very sparse), weights by the opponent's mix, and converts the difference from her career baseline into a bounded edge score.
3. If data is insufficient at any step, the signal reports itself unavailable and contributes exactly 0 — it never guesses.

This signal exists only because of the Wikipedia style enrichment (7.3% to 32.7% bowler coverage) — a concrete example of a data-collection investment paying off downstream.

The target: why WPA and not contribution share

The models originally predicted contribution share (what fraction of the team's runs/wickets a player produced). This was replaced with **WPA** after a concrete failure: contribution share ranked a part-time all-rounder above a proven world-class opener, because share measures *how big your numbers were*, not *whether they moved your team toward winning*. The switch is validated by a sanity check on real data: Smriti Mandhana averages **+0.077 batting WPA per match over 144 matches**, versus **−0.002 over 23 matches** for the player who had wrongly outranked her. Contribution percentages remain useful — as *features*; they are no longer the target.

Models: what is trained, what was tried, what won

The two production models

Two **independent** regressors — one per discipline — rather than one blended target, so a specialist's zero in her other discipline can't drag down the discipline she excels at:

Target	Production model	Key hyperparameters
`batting_wpa`	`RandomForestRegressor` in a preprocessing pipeline	300 trees, `max_depth=10`, `min_samples_leaf=5`, seed 42

Target	Production model	Key hyperparameters
`bowling_wpa`	`VotingRegressor` blending a shallow `HistGradientBoostingRegressor` (`max_depth=2`, `min_samples_leaf=50`) with a regularized RF (`max_depth=6`, `min_samples_leaf=50`)	as listed

Preprocessing: median imputation on numerics (structural missingness — specialists who never bowled — is expected, not dirty data) and one-hot encoding of `role_bucket` with unknown-safe handling. The regularization is not decorative: an unconstrained forest **overfits this noisy target below the mean-predictor baseline**.

Training-label winsorizing: labels are clipped at the 5th/95th percentile for batting and a more aggressive 10th/90th for bowling (whose per-match WPA has fatter tails — one hat-trick over swings it enormously). Crucially, clipping applies only to *training* labels; all reported metrics are computed against **raw, unclipped** holdout labels, so the evaluation stays honest.

Persisted artifacts (`artifacts/player_recommendation/{gender}_{format}/`, e.g. `female_T20/`, `male_T20/`): `model_batting_wpa.joblib`, `model_bowling_wpa.joblib`, and `training_metadata.json` recording the feature list, row counts, date range, and internal holdout metrics — one complete, independent set per `(format, gender)` scope. This directory scoping is itself a fixed bug: before it, every scope shared the same flat filenames, so training a second gender silently overwrote the first's models with no error, and `RecommendationScorer` was calling the no-argument model-loading functions even though it already carried its own `fmt/gender`. Both genders' models are now trained and independently verified present on disk — female: 40,955 rows (40,681 per target after NaN drops), 2009-06-18 to 2026-06-15; male: 71,823 rows (70,273 per target), 2005-02-17 to 2026-06-15.

How training actually works — no epochs, no batches

A common point of confusion worth addressing directly: these are **not neural networks**, so "how many epochs" and "batch size" don't apply here at all — there is no iterative gradient-descent loop over mini-batches of rows. What actually happens, concretely:

- **`batting_wpa` (`RandomForestRegressor`, 300 trees)**: each of the 300 trees is grown from an independent bootstrap resample of the *entire* training set (all ~41K/72K rows at once, not batched), split greedily by the standard CART algorithm down to `max_depth=10` or until a leaf would hold fewer than `min_samples_leaf=5` rows. There is no "loss curve" to watch converge — the forest is built once, in one `.fit()` call, and that's the whole training process.
- **`bowling_wpa` (`VotingRegressor` of `HistGradientBoostingRegressor` + `RandomForestRegressor`)**: the RF half works exactly as above (`max_depth=6`, `min_samples_leaf=50`, more conservative since bowling WPA is the noisier target). The HistGB half is the one component that *does* have an iterative-rounds concept close in spirit to epochs — it fits successive shallow trees, each correcting the residual of the ones before it, up to scikit-learn's default 100 boosting rounds (`max_depth=2`, `min_samples_leaf=50` keeps each round intentionally weak, since deeper/greedier rounds were found to just memorize per-match noise). The two halves' predictions are then simply averaged.
- **Preprocessing** happens once, up front, not per-batch: numeric features are median-imputed, `role_bucket` is one-hot encoded, and *training* labels are winsorized (5th/95th percentile batting, 10th/90th bowling) before either model ever sees them.
- **"Tuning" was evidence-based model/hyperparameter selection, not iterative loss minimization.** The real tuning happened at a different level: comparing whole model families (this RF/HistGB choice vs. plain HistGB, LightGBM, XGBoost, Linear/Ridge) across 3 temporal-holdout fractions × 5 random seeds each, and keeping whichever won on ranking correlation — see "Models

tried" below. There is no hyperparameter grid search or early-stopping schedule to report because none was used; the depth/leaf-size values shown in the table above were themselves picked by checking real holdout numbers (documented in `train.py`), not swept automatically.

In short: "how many epochs" has no real answer here because the training procedure is fit-once-on-everything tree building, not incremental mini-batch learning. If the project later adopts a genuinely iterative model (e.g. a calibrated gradient-boosted win-probability model, floated in "What Can Be Done to Make It Better"), *that* would be the first place an epoch/early-stopping story would actually apply.

Fine-tuning attempt: adding an age feature, and why it wasn't kept

Directly responding to a request to "fine-tune the model… with proper consideration of age factor, active status, ground performance, domestic performance": three of these four were already real model inputs before this pass (`venue_bat_contribution_pct`/`venue_bowl_contribution_pct`/`venue_matches` for ground performance, `domestic_form_batting_wpa`/`domestic_form_bowling_wpa`/`domestic_form_matches` for domestic performance — see the feature schema table above), and the fourth (active/retirement status) is deliberately handled as a **rule-based eligibility gate and score component** (`roster.py`'s exclusion, `score.py`'s `risk_part`), not a raw trained-model input — adding it as a training feature would either leak (registry status reflects *today's* snapshot, not the player's status as of each past training row) or duplicate information the recency-weighted `form_*/career_matches` features already carry, for no clear benefit. Reconstructing a genuinely point-in-time version would cost real engineering effort for a signal the model likely already sees indirectly — a judgment call recorded here rather than silently skipped.

Age was genuinely new and was actually tried. `players.date_of_birth` (partial real coverage from opt-in ESPNcricinfo enrichment — 36.3% of all 7,580 players, but 55.8%/60.6% of actual *training rows* for women's/men's respectively, since enriched players tend to be the more-capped ones who generate more rows) makes a leakage-safe `age_years` feature computable — a player's exact age *as of that specific match's date* is an objective fact, not derived from any outcome, so unlike every other feature in `historical.py` there was no leakage-safety design problem to solve.

It was added, tested with the exact same rigor as every other model decision in this project (3 holdout fractions × 5 seeds, both genders, the real production model architecture, real live data, metrics against raw unclipped labels), and **honestly did not survive the check**:

Gender / target	Correlation, without age	Correlation, with age	Verdict
female / batting_wpa	0.12076	0.12107	negligible gain (within noise, std 0.005)
female / bowling_wpa	0.07815	0.07810	negligible loss
male / batting_wpa	0.08625	0.08496	**worse**
male / bowling_wpa	0.06500	0.06413	**worse**

MAE was identical to 4 decimal places in every case. Age made real predictive performance measurably *worse* in two of the four (gender, target) combinations and negligibly different (within run-to-run seed noise) in the other two — no combination showed a real improvement. Following this project's own established discipline for exactly this situation (see "Experiments tried and rejected" below — bowling containment features and `bowl_style_bucket` were both rejected the same way, on the same kind of evidence), `age_years` was **not** added to `MODEL_FEATURES`. It's still computed at live-scoring time and shown as descriptive-only narrative color in `explain` output ("Left-handed balanced, 30 years old.") — genuinely useful context for a human reader, with none of the risk of feeding a null result back into a prediction.

Models tried — the full comparison

The robust comparison notebook (`model_comparison_robust.ipynb`, results in `model_comparison_robust.csv`, 223 result rows) evaluated, per target:

- Mean-predictor baseline (predicts the training mean for every row)
- The production model (imported directly from `train.py`, so no configuration drift)
- Linear Regression and Ridge (with standard scaling)
- A generic Random Forest (same 300/10/5 config, seed-varied)
- HistGradientBoosting (sklearn defaults)
- LightGBM (300 trees, depth 6, learning rate 0.05)
- XGBoost default (300 trees, depth 6, lr 0.05) and XGBoost tuned (depth 3, lr 0.05, subsample 0.8, column subsample 0.8)

Batting WPA (baseline MAE 0.0812), mean over 3 holdout fractions x 5 seeds, sorted by correlation:

Model	MAE	R^2	Correlation (mean ± std)
Random Forest (generic)	0.0819	0.0181	0.1375 ± 0.0025
Production (train.py)	0.0819	0.0175	**0.1350 ± 0.0009**
XGBoost (tuned)	0.0822	0.0171	0.1317 ± 0.0041
HistGB (generic)	0.0817	0.0161	0.1296 ± 0.0067
LightGBM	0.0829	0.0129	0.1183 ± 0.0047
XGBoost (default)	0.0833	0.0088	0.1076 ± 0.0021
Ridge / Linear	0.0829	0.0110	0.1069 ± 0.0040

Bowling WPA (baseline MAE 0.0677):

Model	MAE	R^2	Correlation (mean ± std)
Production (train.py)	0.0685	0.0091	**0.1111 ± 0.0063**
XGBoost (tuned)	0.0688	0.0104	0.1069 ± 0.0098
Random Forest (generic)	0.0686	0.0088	0.0985 ± 0.0081
HistGB (generic)	0.0685	0.0079	0.0955 ± 0.0094
Linear / Ridge	0.0693	0.0073	0.0910 ± 0.0045
LightGBM	0.0693	0.0047	0.0768 ± 0.0037
XGBoost (default)	0.0696	0.0015	0.0674 ± 0.0146

Why the current models were selected

- **Batting: keep the production Random Forest.** The "generic RF" that tops the table *is the same configuration* — the tiny gap is seed variation. HistGB has marginally lower MAE but captures less directional signal, and **ranking is what the engine actually does with these predictions**, so correlation is the decision metric.
- **Bowling: keep the production Voting ensemble.** It wins outright on correlation with the **lowest run-to-run variance**. Tuned XGBoost is the closest challenger (slightly better R^2) but loses on correlation and is noticeably noisier — it is flagged as the configuration to re-examine if the feature set changes.
- A sobering, honestly-reported observation: **every real model is slightly worse than the mean predictor on MAE.** For a target this noisy that is expected — the models' value is in *ranking* players (correlation, hit rates), not in absolute per-match error.

Why the comparison is called "robust"

A single evaluation can lie in two ways, and the methodology defends against both:

1. **A lucky/unlucky split** — so it uses *three* temporal holdout fractions (15%, 20%, 25%) instead of one.
2. **A lucky/unlucky random seed** — so every stochastic model is run with *five seeds* per fraction, reported as mean ± standard deviation.

It reports MAE, R^2, and correlation together and sorts by correlation because ranking is the deployment task. One recorded caveat: the production model's standard deviation covers only 3 runs (its seed is fixed by design so the shipped configuration stays deterministic), so it is not a perfectly apples-to-apples seed sweep against the 15-run challengers.

Experiments tried and rejected (with reasons)

Experiment	Outcome
Bowling containment features (`bowl_dot_pct`, `bowl_boundary_pct`, `bowl_strike_rate`) as model inputs	**Rejected** — every variant made bowling-WPA prediction worse (correlation 0.103 down to as low as 0.088). They survive as descriptive color in explanations only.
Coarse `bowl_style_bucket` (pace/spin/unknown) categorical feature	**Rejected** — no measurable difference across the full 3x5 evaluation grid.
XGBoost as the production family	**Rejected** — beaten by the shipped models on the ranking metric on both targets.
Learning-to-rank (LGBMRanker, lambdarank) instead of regression	**Rejected** — regression-based ranking beat it on both ranking metrics (top-11 hit rate 0.626 vs 0.579; Spearman 0.28 vs 0.116). The likely cause: collapsing WPA into coarse relevance grades throws away role-specific signal. Kept in the repo as a runnable comparison.
Unregularized Random Forest	**Rejected** — overfits below the mean-predictor baseline on this noise level.
`age_years` (player's age at each match's date) as a model feature	**Rejected** — worse correlation for both men's targets, negligible/noise-level difference for both women's targets, MAE unchanged to 4 decimals, across the full 3x5 evaluation grid, both genders. Survives as descriptive color in explanations only (`current.py`/`explain.py`), the same treatment as the bowling containment features above.

The pattern worth noting: each rejection is recorded *with its measured evidence*, so future contributors don't re-litigate settled questions — and know exactly which numbers to beat if circumstances change.

Evaluation: metrics and validation strategy

Metrics, defined

Regression backtest (`evaluate/backtest.py`), per target on the temporal holdout:

- **MAE (mean absolute error)** — average of |prediction − actual|. Intuitive units (WPA per match); insensitive to outliers relative to squared errors.
- **Baseline MAE (mean predictor)** — the MAE you'd get by always predicting the training-set mean. Every model must be read against this bar; it is the honest floor.
- **R^2 (coefficient of determination)** — the fraction of the target's variance the model explains. Values near zero here (0.005–0.018) quantify just how noisy single-match WPA is.
- **Pearson correlation (predicted vs actual)** — measures *directional/ranking* signal even when absolute errors are large; the primary decision metric, because the engine consumes predictions as rankings.
- **Top-quartile hit rate within match** — within each held-out match (with at least 4 players), the fraction of the *actual* top-25% performers that appear in the *predicted* top-25%. A deliberately modest proxy for selection quality: random guessing scores ~25%; the models score ~32% (batting) and ~36% (bowling). It only checks identification of standouts among players who actually played — reconstructing "XI versus full squad" would require bench-player features the data doesn't provide.

Ranking backtest (`evaluate/rank_backtest.py`), per held-out match:

- **Top-11 hit rate** — of the actual top 11 players by real combined WPA, how many appear in the predicted top 11.
- **Mean Spearman rank correlation** — agreement between the predicted ordering and the actual ordering within each match.
- **NDCG was deliberately avoided:** its exponential gain formula (2^relevance − 1) misbehaves for a continuous, frequently *negative* quantity like WPA.

Win-probability model (in data-collection): **Brier score** (mean squared error of the probability, ~0.17 vs 0.25 baseline) and **log-loss** (~0.51 vs 0.69), both on a match-level holdout.

Validation strategy: temporal holdout, not cross-validation

The split is by **match date quantile** (train strictly before the cutoff; test at/after; default test fraction 20%). Standard k-fold cross-validation is unsuitable here for two reasons: rows within a match are correlated (random folds leak results across the boundary), and shuffled folds let the model train on the *future* and predict the *past*, which inflates scores relative to the real deployment setting (always predicting forward in time). The backtest also imports the *same* model factory and clipping function the production trainer uses, so the reported numbers describe the shipped model, not a lookalike.

The recommendation engine

The composite score

`RecommendationScorer` produces a 0–100 score as a **documented sum of parts**, each rounded, bounded, and carrying an `inputs` dictionary of the raw values behind it:

Component	Range	What it measures
`model_part`	0 to 45	Predicted batting + bowling WPA (or a shrinkage fallback below 10 career matches), scaled by 450. The player's stronger discipline counts at full weight; the weaker one at 0.4 — so all-rounders don't structurally outscore specialists.
`matchup_part`	−21 to +21	Three sub-signals: aggregate batter-vs-this-opponent dismissal record (±8, shrunk, prior strength 48); the style-matchup edge (±5); and the symmetric bowler-side record against the opponent's *current* batters (±8, opposite polarity — a bowler above her career wicket rate against them is a plus). The bowler side closed a structural gap where specialist bowlers received almost no matchup credit.
`phase_fit_part`	0 to 8	How strong the player's best batting phase percentile and best in-team bowling phase rank are.

Component	Range	What it measures
`standing_part`	0 to 20	Track record: the importance score (scaled) plus a bonus for holding an ICC top-10 ranking.
`risk_part`	0	Penalties only: −3 age advisory, −5 INCONSISTENT selection history, −4 INACTIVE-but-recalled.
`team_need_part`	0 to 5	Pool-level synergy bonus (below).

Cold start: with fewer than 10 career matches, the learned model is skipped entirely; career WPA shrunk toward scope-wide priors substitutes, and the output records *why* (`model_applied=false` plus a reason string) rather than silently degrading.

Calibration honesty: several constants (the 450 WPA scale, the ±8 matchup multipliers ×120/×175, the team-need ×35) are *calibrated mappings* — chosen against the real standard deviations of the underlying quantities and validated by eyeballing output — not fitted hyperparameters. The first team-need scale (×200) maxed the bonus out for nearly everyone and was caught precisely by that eyeballing.

Team-need synergy

`compute_team_need` finds the pool's single weakest phase (powerplay / middle / death) by comparing the pool's average combined per-phase WPA against a benchmark, then `player_team_need_bonus` rewards players whose per-phase record *exceeds* the benchmark in exactly that phase — up to +5, never a penalty. So if the squad is collectively weak at the death, proven death-overs performers get a nudge. The benchmark is scoped to the pool's own gender (fixed — it used to blend both genders' `player_phase_wpa` into one format-wide number; real benchmarks now measurably differ, e.g. death-overs benchmark −0.001 for men vs. +0.005 for women on live data). Since `player_phase_wpa` has no gender column of its own, scoping goes through each player's real match participation (`player_match_wpa` join `matches.gender`), the same technique used throughout the codebase wherever `players.gender` (only ever set for the ~32 split-collision identifiers) isn't available directly.

Explanations

`explain.py` converts the component breakdown into coach-facing sentences: a headline rating, the slot context ("Picked for the opening slot ahead of X" or "an uncontested pick"), the model or cold-start basis expressed as per-match win-probability percentages, a descriptive bowling profile (control-focused / attacking / balanced, from dot and boundary percentages), each matchup signal, phase fit, team need, standing, and risk flags. The hard rule: explanations **never state anything that isn't in the computed numbers**.

Player analysis: what to expect, and where the impact is. Beyond the score components, `explain.py` surfaces the descriptive batting profile from `player_advanced` (plumbed through `score.py`'s `inputs`, never a model feature): archetype + hand + age as a headline ("Right-handed finisher, 23 years old"), a "what to expect" line (strike rate, boundary%, dot%, sixes and six rate), and a "highest impact" line naming the phase where the batter's strike rate peaks with that phase's SR and boundary%. For a genuine finisher/power hitter whose composite rating is nonetheless low, it adds an explicit transparency note — the rating is win-probability-added, not raw hitting, so a big-hitter with modest recent win-impact rates low, worth a manual look. This directly answers the recurring "she's our best finisher but isn't recommended" question (real case: RM Ghosh, death-overs strike rate 166, 45 sixes, rating 7.8).

Specialist role options. `recommend` also emits, separately from the score-ranked shortlist, the best available finisher / power hitter / anchor across the *whole* scored pool (`selection/optimize.role_highlights`), ranked by that role's *defining* metric (finisher by death-overs strike rate, power hitter by six rate) rather than the composite score — otherwise the win-probability rating would re-bury the very player the coach

is looking for. So RM Ghosh surfaces as the side's best finisher even while sitting at 7.8/100 outside the auto-XI.

House style. These sentences deliberately use no em-dash / en-dash connectors ("—"/"–"); commas, colons, and parentheses instead, per a maintainer preference. Ordinary hyphens inside words ("all-rounder", "Right-handed", "vs-pace") are unaffected. Both `explain.py` and the `render.py` team-sheet headers follow this, with a regression test asserting no dash connectors appear in generated reasoning.

XI selection as constrained optimization

Picking the best XI is not "take the top 11 scores" — a legal team needs structure. `selection/optimize.py` formulates it as a **0/1 mixed-integer linear program** (via `scipy.optimize.milp`): maximize total score subject to exactly 11 players, at least 5 batting-capable, at least 5 bowling-capable, at least 1 wicket-keeper, and at least 2 openers (median batting position 2.5).

Three design details matter:

1. **Constraints degrade gracefully.** Each minimum is capped at how many qualifying players actually exist in the pool — a pool with no keeper data drops the keeper constraint to zero instead of becoming infeasible.
2. **The MILP runs over the entire candidate pool**, not the top-20 display shortlist. A required keeper or opener often scores below the shortlist cutoff; truncating first would silently violate constraints. This was a real bug, now guarded by regression tests.
3. **A greedy fallback** exists if the solver fails: swap the best benched qualifying player for the worst non-qualifying XI member until each quota holds.

A separate presentation step (`_team_sheet_order`) then arranges the *selected* XI like a real batting card — best two openers in slots 1–2, batting core by merit, specialist bowlers in the tail — keeping selection (an optimization problem) cleanly apart from ordering (a presentation concern).

Performance note

Scoring a full pool with explanations originally took ~78 seconds, dominated by recomputing the opponent's roster once per scored player. A per-opponent roster cache cut this to ~23 seconds.

Testing (model-training)

A 10-file pytest suite runs entirely against a tiny in-memory SQLite fixture (built in `conftest.py` with helper builders — `register_player`, `add_match`, `add_match_impact`, and so on) that mimics only the columns the module reads. No dependency on the real database. The most important guarantees under test:

- **Leakage safety** — career/WPA features at row N are provably built from rows 1..N−1 only.
- **Shrinkage behaviour** — zero-sample opponent/venue records collapse exactly to the career prior.
- **Eligibility** — most-recent-appearance team derivation, retirement exclusion, and the cross-gender collision resolving to two distinct players.
- **Scoring semantics** — components sum exactly to the total; cold start bypasses the model; dot/boundary percentages appear in narrative only (never as score); the WPA-target ordering (proven match-winner beats flashy fringe player); team-need applied by pool scoring but not solo scoring.
- **Selection** — quota satisfaction, keeper/opener rescue from outside the display cutoff, greedy fallback, and team-sheet ordering.
- **Rendering** — both sections present, every pool member shown with reasoning, XI tagging.

Not unit-tested (exercised via CLI and notebooks instead): actual model fitting, the two backtests (they need a large real database), and CLI argument wiring.

Strengths and Weaknesses — an Honest Assessment

What the project does well

- **Correctness under real-world data mess.** The cross-gender identifier collisions, the recall-flag bug, the wickets-dilution mask, the venue-cliff shrinkage fix — each was found against live data, fixed at the root, and pinned with a regression test. The willingness to rebuild a table's unique constraint (with a backup for verification) rather than patch around it is unusual discipline for a project this size.
- **Leakage-safe by construction and by test.** Point-in-time feature reconstruction with unit tests that assert strict pre-match provenance, plus temporal-only evaluation splits. Many far larger projects get this wrong.
- **Honest evaluation culture.** Every model is compared against a mean-predictor floor; metrics are computed on raw (unclipped) labels; weak absolute numbers are stated plainly; the robust comparison controls for split luck and seed luck; rejected experiments are recorded with their measured evidence.
- **Transparency end to end.** From `importance_score` components in the database to the additive recommendation score with an `inputs` dictionary per component, every number a coach sees can be traced to raw data. Explanations never exceed the numbers.
- **A clean module contract.** One read-only file between the halves; either side testable in isolation against synthetic fixtures.
- **Ethical data sourcing.** Per-host rate limits, an identifying User-Agent, refusal to bypass ICC credentials, no scraping of pages that forbid it, no fabricated metrics, and human confirmation for retirement news.
- **Idempotent, resumable operations.** Fingerprinted downloads, quarantined rejects, upserts everywhere, per-stage CLI re-runs.

What is weak or unfinished

- **The predictive signal is genuinely small.** Correlations of ~0.14 (batting) and ~0.11 (bowling), R^2 near zero, and MAE *slightly worse* than the mean predictor. Single-match WPA is dominated by variance the features cannot see (conditions, luck, opposition plans). The engine compensates by leaning on career/form aggregation and rule components — reasonable, but the "model" part of the score is currently a weak signal amplified by careful framing.
- **Style/keeper coverage is partial, and gendered.** Bowling styles cover ~33% of real female T20 bowlers and a comparable ~43% of male bowlers (confirmed on live data — the enrichment itself was never gender-restricted). Wicket-keeper detection, by contrast, covers 18 **women's** T20I nations' current squads and **zero** men's nations — that research was only ever done for one gender. Outside whatever coverage exists, the corresponding signals silently contribute nothing (correctly, but nothing) — recommending for an uncovered men's team currently gets no keeper-slot preference, gracefully degraded rather than broken.
- **Two pipeline stages are easy to forget.** `wpa` and `role-finalize` are not part of `refresh`; an automated schedule that runs only `refresh` serves stale WPA labels and roles to the model half.
- **Some constants are eyeballed.** The scaling constants in the composite score are calibrated by inspection, not fitted or sensitivity-tested. They are documented, which mitigates but does not remove the arbitrariness.
- **Win probability v1 is coarse.** Step-shaped bins, no team-strength or conditions prior. Since WPA (the training label!) derives from it, label quality is bounded by WP-model quality — an important, easily-missed coupling. (It is, as of the gender-isolation audit, at least fit separately per gender rather than blended — see below.)
- **The production seed-sweep caveat.** The shipped models' variance estimate rests on 3 runs versus 15 for challengers.
- **Single-format horizon, within each gender's domestic scope.** ODI ingestion is scaffolded but disabled for both genders. Women's domestic T20 leagues (WBBL, WPL, The Hundred, …) are ingested and feed a `domestic_form` feature; the five scaffolded men's domestic leagues (IPL, BBL, PSL, CPL, NTB) remain `enabled=False` — men's T20I recommendations currently work from international data only, with no domestic-form signal. The webapp integration is still ahead.

Resolved since the above was first written (kept for the record)

A later, dedicated audit — prompted by a direct request to verify "men's and women's data are never mixed… for all country" and that venue analysis is correct — found and fixed several real bugs beyond what the original gender migration caught:

- `player_card.py` blending/dropping cards across the 32 split identifiers (most severe: silently summed both genders' careers into one card).
- `h2h.py`'s (and `phases.py`'s `phase_vs_opponent`'s) raw-name resolution blending H2H stats across the same 32 identifiers.
- The win-probability model itself being fit on both genders' balls blended together — the most foundational of the fixes, since almost every WPA feature/training label derives from it.
- `styles.py`'s dormant (not yet triggered, but unsafe) gender-blind style lookup.
- `team_need.py`'s format-wide (gender-blended) benchmark.
- Two genuinely different real grounds ("County Ground", "National Stadium") silently sharing one `venue_id`.

Zero `player_id` overlap between men's and women's eligible pools was independently confirmed across **all 89 countries** with both genders' T20I history in the live database, both before and after these fixes. Full detail, including real before/after numbers for each, lives in both packages' READMEs.

What Can Be Done to Make It Better

Ordered roughly by expected value:

1. **Improve the label before improving the model.** WPA quality is capped by the win-probability model. Replacing the empirical grid with a calibrated ML model (the `fit`/`predict` surface was designed for exactly this drop-in) that knows team strength, venue scoring norms, and era effects would sharpen every downstream label and likely raise the ceiling on all model metrics at once — now doubly true since the grid is fit per gender, so this upgrade would need to respect that same split.
2. **Automate the out-of-band stages.** Fold `wpa` and `role-finalize` into `refresh` (or a `refresh --full`), so scheduled runs can't silently serve stale labels.
3. **Richer context features.** Venue scoring environment (average first-innings score, chase success rate), opposition strength at match time, tournament stage, and days-since-last-match are all derivable from data already in the database and are the most plausible sources of new signal.
4. **Uncertainty estimates.** Quantile regression or forest prediction intervals would let the engine distinguish "confidently mediocre" from "high-variance wildcard" — directly useful for selection decisions and honest explanations.
5. **A true seed sweep for the production configuration**, and a small sensitivity analysis over the calibrated scale constants (WPA scale, matchup multipliers, team-need scale), replacing eyeballed values with checked ones.
6. **Close the coverage gaps.** Continue Wikipedia/Wikidata style enrichment; extend wicket-keeper detection to men's nations (currently zero coverage — a data-research gap, not a code one, using the exact same `role-finalize --load-external` mechanism already proven for 18 women's nations); pursue a legitimate full ICC rankings source.
7. **Enable men's domestic T20 leagues.** IPL/BBL/PSL/CPL/NTB are scaffolded (`enabled=False` in `config.py`) but never ingested — women's domestic leagues already feed a real `domestic_form` feature; men's currently has no equivalent signal.
8. **Revisit learning-to-rank properly** — with role-specific relevance labels rather than a single collapsed grade, LTR's earlier loss to regression may not hold.
9. **Operational hardening.** CI running both test suites, a scheduled refresh with freshness alerts, and database schema versioning would move the project from "runs on my machine" to "runs unattended".
10. **Serve it.** The webapp exists as a stub; the render/explain layer was built to be consumed by something friendlier than a terminal.

Items that were on this list and are now done, kept for the record: fixing the `h2h.py`/`player_card.py` gender-resolution gap; scope-consistent (per-gender) team-need benchmarks; extending to men's T20I (trained, verified, documented above) — each closed by the gender-isolation audit described in the previous section.

Appendix A — Dependencies and Why Each Exists

data-collection

Package	Why
requests	All HTTP (downloads, APIs) with the shared polite session and retries.
pandas	Ingest shaping and every derived-table builder.
pyarrow	Parquet: ball-state and win-probability datasets, model grids, enrichment cache.
rapidfuzz	Fuzzy name matching (fallback only; threshold 88–90).
feedparser	RSS retirement-feed scanning.
beautifulsoup4	Declared for HTML parsing (lightly used — Wikipedia parsing is regex-on-wikitext; ESPN uses the JSON API).
matplotlib, jupyter, ipykernel, nbformat, nbconvert	Notebooks and their execution.
pytest	Test runner.

Standard library deliberately leaned on: `sqlite3`, `zipfile`, `csv`, `uuid`, `argparse`, `logging`, `json`, `datetime`, `zlib`, `re`, `bisect`, `urllib`.

model-training

Package	Why
numpy	Array math (feature weighting, MILP masks, metrics).
pandas	SQL-to-DataFrame reads; group-wise expanding/rolling feature construction.
scikit-learn	Model families (RandomForest, HistGradientBoosting, Voting), pipelines, imputation, encoding, splits.
scipy	`optimize.milp` for XI selection; `stats.spearmanr` for the rank backtest.
joblib	Model artifact persistence.
pyarrow	pandas 3.x columnar backend.
matplotlib	Comparison-notebook plots.
xgboost, lightgbm	Challenger models in the comparisons (LightGBM also powers the optional `rank-backtest`).
jupyter, ipykernel, nbformat, nbclient	Notebook execution.
pytest	Test runner.

Appendix B — CLI Quick Reference

Terse pointers only — for full detail on every command (arguments, exactly what each reads/writes, worked examples), see `docs/cli-reference.md`.

data-collection (`python run.py <cmd>`)

Command	Purpose
`refresh [--force] [--enrich] [--seed-styles] [--no-rss]`	Full 12-stage pipeline.
`init`, `download`, `ingest`, `identity`, `registry`, `cards`, `career`, `advanced`, `opponents`, `impact`, `venues`, `matchups`, `phases`, `ml`, `wpa`	Individual stages.
`enrich [--limit N] [--refetch]`	ESPN JSON enrichment.
`styles [--scaffold|--seed|--wiki|--load] [--all] [--limit N]`	Bowling-style workflows.
`rankings [--scaffold|--fetch]`, `status-override [--scaffold]`	Pluggable CSV workflows.
`role-finalize [--format T20] [--load-external CSV]`	Role refinement + keeper merge (women's-only coverage so far).
`rss`, `alerts [--confirm N|--dismiss N]`	Retirement monitoring (human-in-the-loop).
`winprob [--format T20] [--gender female] [--match ID] [--state "..."]`	Win-probability build (per gender) / query.
`status`, `card`, `profile`, `sheet`, `h2h`, `vs-team`	Inspection and selector reports.

model-training (`python -m criiq_models.cli <cmd>`)

Every command below also takes `--format T20` and `--gender female|male` (default `female`).

Command	Purpose
`features`	Build the leakage-safe training table (row count + date range).
`train`	Fit and persist both WPA models + metadata for this `(format, gender)` scope.
`backtest`	Temporal-holdout regression report.
`rank-backtest`	Learning-to-rank vs regression comparison.
`teams`	List known teams.
`recommend --team A --opponent B [--venue V] [--explain] [--pretty]`	Score the pool, select the XI, explain, render.
`explain --player-id ID | --name NAME --opponent B`	Single-player component breakdown.
`analyze-xi --team A --opponent B (--players "..." | --player-ids "...")`	Score + check a coach's own hand-picked XI against the same constraints and the AI's optimal pick.

Appendix C — Glossary

Term	Meaning
WPA (Win-Probability-Added)	The sum over a player's balls of the change in her team's win probability; striker credited +delta, bowler −delta per ball. The models' prediction target.
Contribution share	The fraction of a team's output a player produced. Former target; now a feature only.
Shrinkage	`(n*observed + k*prior)/(n + k)` — pulls small-sample statistics toward a baseline.

Term	Meaning
Leakage	Any path by which information from the predicted match (or its future) enters that row's features; prevented by point-in-time reconstruction and temporal splits.
Temporal holdout	Train on matches before a date cutoff, test on matches after it.
Winsorizing	Clipping extreme label values (training only) so single freak performances don't dominate the fit.
MILP	Mixed-integer linear program — the exact-optimization formulation of XI selection under composition constraints.
Cold start	A player below 10 career matches: model skipped, shrinkage baseline substituted, reason recorded.
Bogey bowler	The opponent bowler who has dismissed a given batter most often (minimum 6 balls faced).
Brier score	Mean squared error of a probability forecast (win-probability model quality).
MAE / R^2 / Pearson / Spearman	Mean absolute error; variance explained; linear correlation of predictions with actuals; rank-order correlation.
Registry statuses	ACTIVE, INCONSISTENT (in-and-out of selection), INACTIVE, UNKNOWN, RETIRED_FORMAT, RETIRED_ALL.
Cricsheet	The open ball-by-ball cricket data project; Criiq's authoritative event source.